CSE 31
Computer Organization

Lecture 13 – Integer Representation (wrap up) and
MIPS Assembly Language

Announcements

- Lab
  - Lab 4 due this week (with 7 days grace period* after due date)
    - » Demo is REQUIRED to receive full credit**
  - Lab 5 out *next* week
    - » Due at 11:59pm on the same day of your lab after next (with 14 days grace period^ after due date)
    - » You must demo your submission to your TA within **21 days from posting of lab**
    - » Demo is REQUIRED to receive full credit**
- Reading and Homework assignments
  - Reading 03 (zyBooks 3.1 – 3.7, 3.9) due, 09-MAR
  - Homework 02 (zyBooks 2.1 – 2.9) due, 09-MAR
    - » Complete Participation Activities in each section to receive grade towards Reading assignments (part of course participation grades)
    - » Complete Challenge Activities in each section to receive grade towards Homework assignments
    - » IMPORTANT: Make sure to submit score to CatCourses by using the link provided on CatCourses

* There is a 10% penalty for late submissions during the grace period

^ There is NO PENALTY for submissions during the grace period for this assignment

** Submissions without demo will only receive partial credit for the TPS Activity portion of the assignment

Announcements

- Project 01
  - Due Fri, 20-MAR
  - Can work in teams of 2 students
    - » Each team member must identify teammate in "Comments…" text-box at the submission page
    - » If working in teams, each student must submit code (can be the same as teammate) and demo individually
    - » Grade can vary among teammates depending on demo
  - Demo required for project grade
    - » No partial credit for submission without demo
  - No grace period
    - » Must complete submission and demo by due date
- Midterm 01
  - On 12-MAR during class
  - See Announcements 07 and 08 on CatCourses for details

Standard Negative # Representation (review)

- Problem is the negative mappings "overlap" with the positive ones (the two 0s). Want to shift the negative mappings left by one.
  - Solution! For negative numbers, complement, then add 1 to the result
- As with sign and magnitude, & one's complement, leading 0s → positive, leading 1s → negative
  - 000000...xxx is ≥ 0, 111111...xxx is < 0
  - except 1…1111 is -1, not -0
- This representation is *Two's Complement*
- This makes the hardware simple!

In C: short, int, long, intN_t (C99) are all signed integers.

Two's Complement Formula

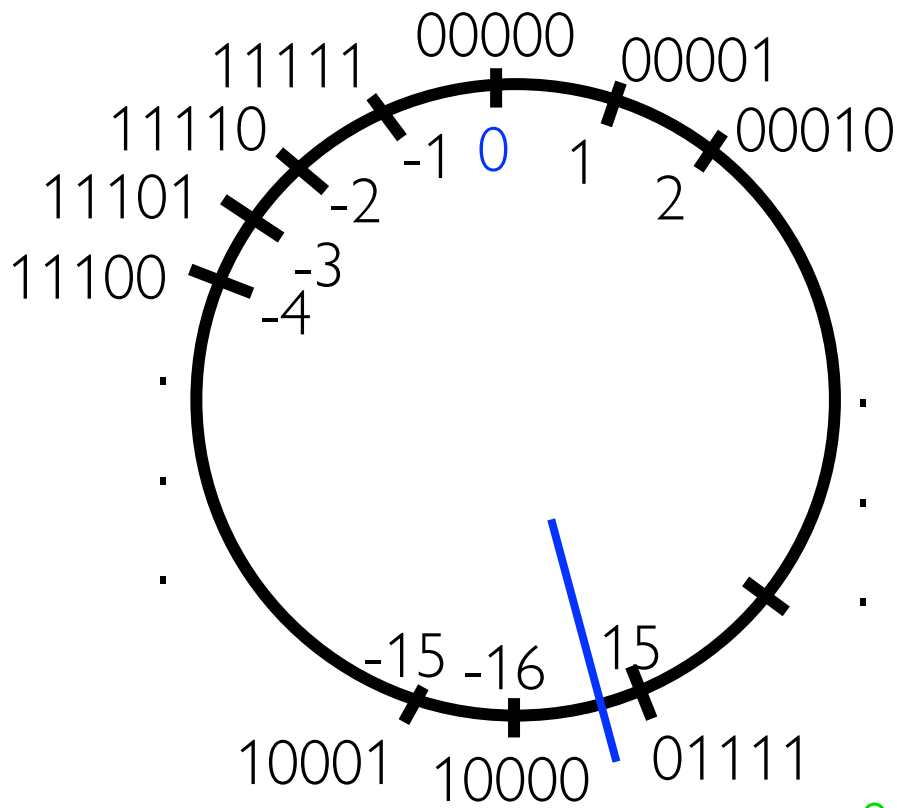
- Can represent positive and negative numbers in terms of the bit value times a power of 2:
  $d_{31} * -(2^{31}) + d_{30} * 2^{30} + \ldots + d_2 * 2^2 + d_1 * 2^1 + d_0 * 2^0$

- Example: 1101_{two}
  $= 1 \times -(2^3) + 1*2^2 + 0*2^1 + 1*2^0$
  $= -2^3 + 2^2 + 0 + 2^0$
  $= -8 + 4 + 0 + 1$
  $= -8 + 5$
  $= -3_{ten}$

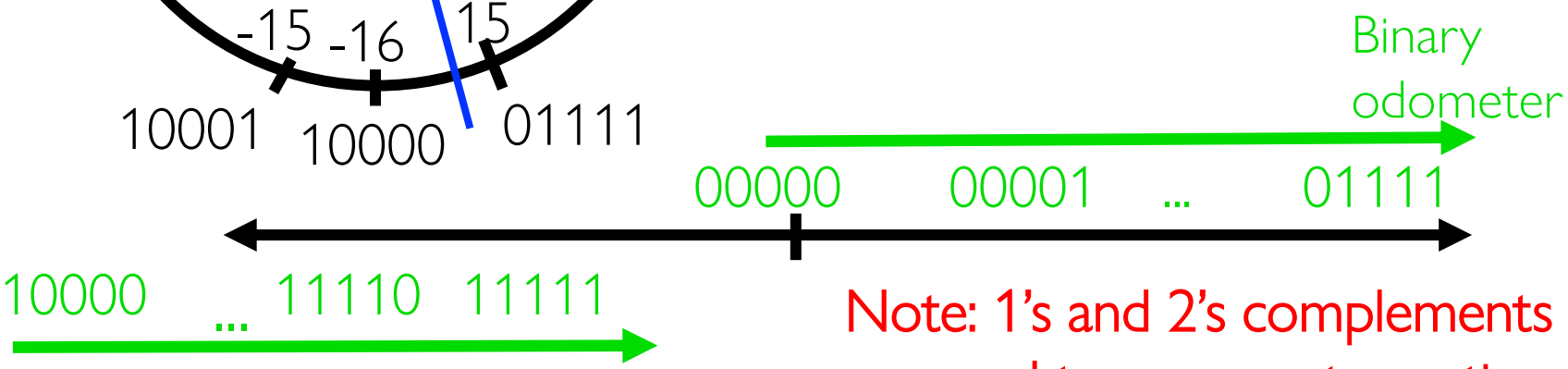
Example: -3 to +3 to -3:

x :	1101_{two}	(-3)
x':	0010_{two}	
+1:	0011_{two}	(3)
()':	1100_{two}	
+1:	1101_{two}	(-3)

2's Complement Number "line": N = 5

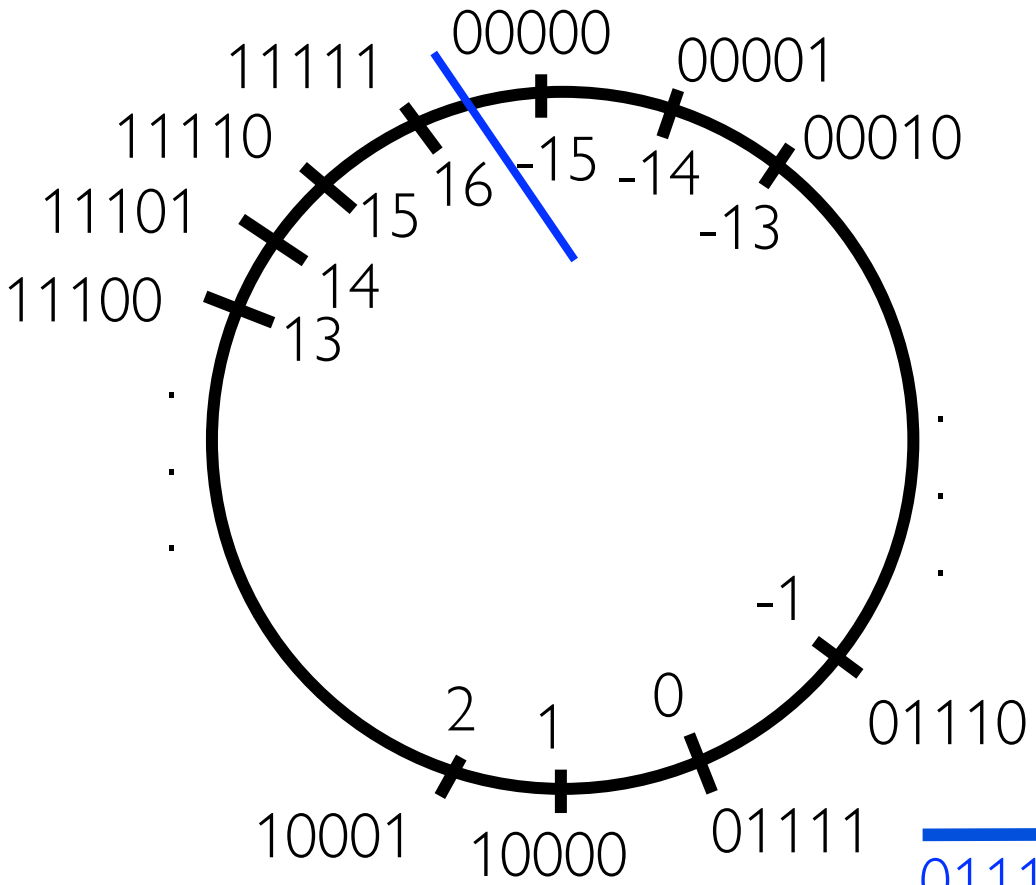


- 2^{N-1} non-negatives (includes zero)
- 2^{N-1} negatives
- one zero
- how many positives?
  - $2^{N-1} - 1$

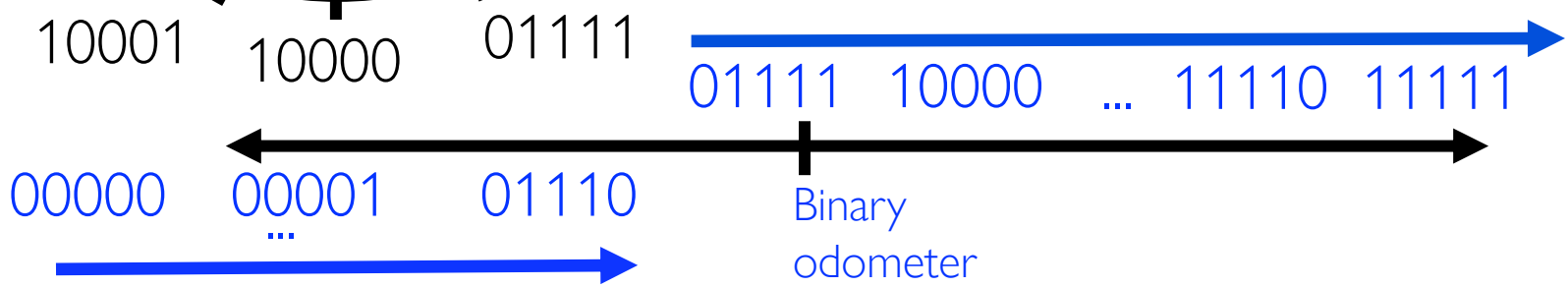


Note: 1's and 2's complements are used to represent negative numbers only!

Bias Encoding: N = 5 (bias = 15)



- Want 00… to represent the smallest number
- value = unsigned - bias
- Bias for N bits = $2^{N-1} - 1$
- one zero
- how many positives?
  - 2^{N-1}
  - (more than 2's complement)



Bias Encoding

- Binary to decimal
  - Convert binary value to decimal (as usual) to get the *unsigned value*
  - Subtract bias from unsigned value to give the final positive or negative decimal value

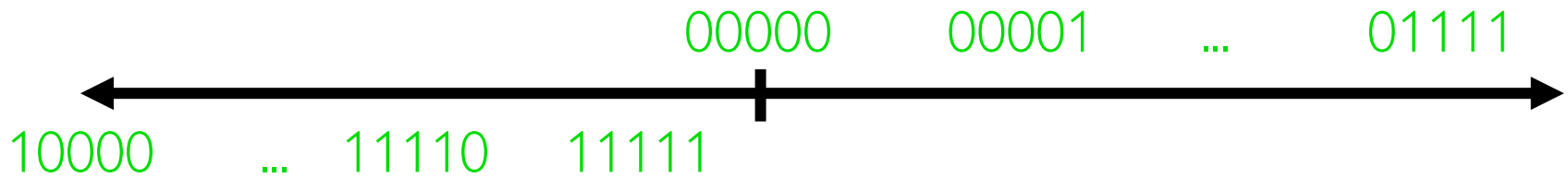
- Decimal to binary
  - Add bias to given value (positive or negative)
  - Then convert the sum to binary

- Unlike Sign Magnitude, 1s complement and 2s complement, positive numbers start with 1 and negative numbers start with 0

Summary

- We represent "things" in computers as particular bit patterns:
  - N bits → 2^N things
- Different integer encodings have different benefits; 1s complement and sign/mag have most problems.
- unsigned (C99's `uintN_t`) :

- 2's complement (C99's `intN_t`): universal, learn it!



- Overflow: numbers ∞ ; computers finite → errors!

We've been *representing* integers. Now let's *register* them in MIPS

Assembly Language

- Basic job of a CPU: execute lots of instructions.
- Instructions are the primitive operations that the CPU may execute.
- Different CPUs implement different sets of instructions. The set of instructions a particular CPU implements is an Instruction Set Architecture (ISA).
  - Examples: Intel x86, IBM/Motorola PowerPC (Macintosh), MIPS (Microprocessor without Interlocked Pipeline Stages), Intel IA64, ...

Instruction Set Architectures

- Early trend was to add more and more instructions to new CPUs to do elaborate operations
  - VAX architecture had an instruction to multiply polynomials!

- RISC philosophy (Cocke IBM, Patterson, Hennessy, 1980s)
  - Reduced Instruction Set Computing
  - Keep the instruction set small and simple, makes it easier to build fast hardware.
  - Let software do complicated operations by composing simpler ones.